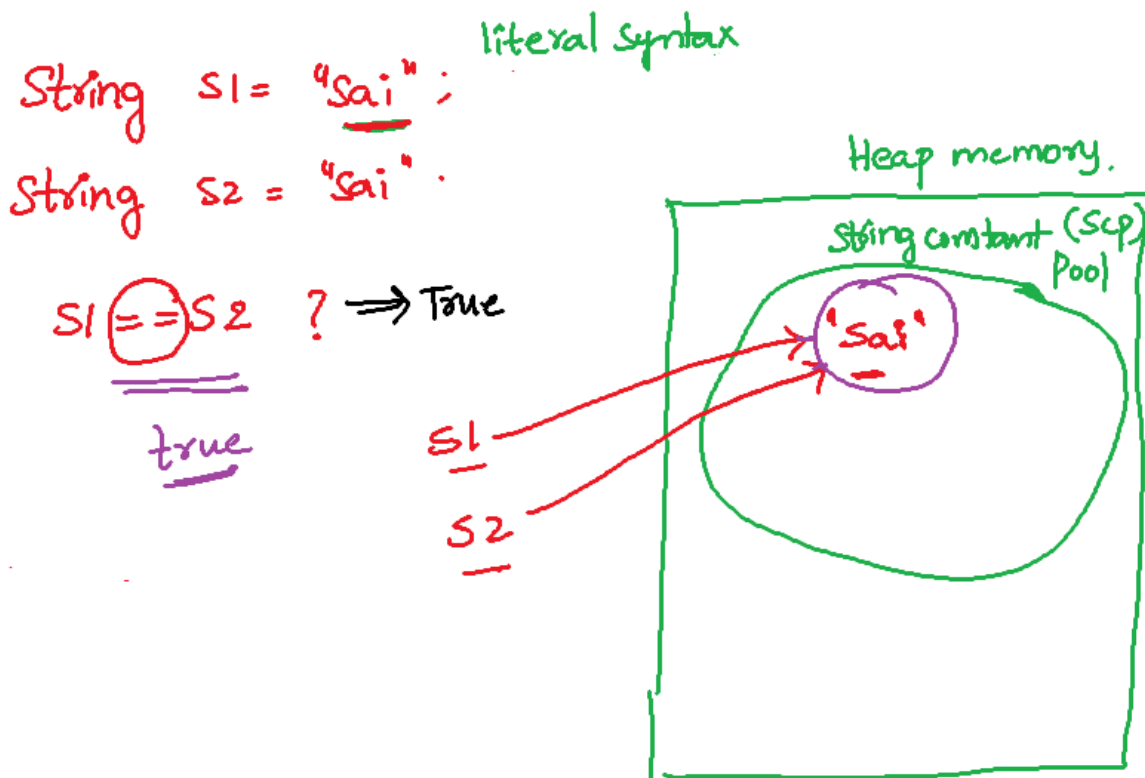


## Part 1 - Guess the output ?

```
String s1 = "Sai";  
String s2 = "Sai";  
  
System.out.println(s1 == s2);
```

Output: true



## Explanation

Before understanding this example, you need to know **how memory is allocated for strings in Java**.

In Java, objects are created inside the **Heap Memory**. Inside the heap, there is a special memory area called the **String Constant Pool (SCP)**.

The String Constant Pool stores string literals and allows Java to reuse existing string objects instead of creating duplicate objects. This optimization helps save memory.

## Step 1: Execute the first line

```
String s1 = "Sai";
```

Since we are creating the string using the **string literal syntax**, the JVM first checks whether the string `"Sai"` already exists in the **String Constant Pool**.

- If the string is **not present**, the JVM creates a new string object `"Sai"` inside the String Constant Pool.
- The JVM then returns the reference of that object.
- The variable `s1` stores this reference and points to that string object.

```
String Constant Pool
```



## Step 2: Execute the second line

```
String s2 = "Sai";
```

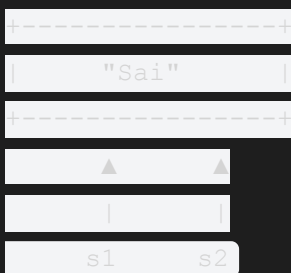
Again, since we are using a string literal, the JVM checks the **String Constant Pool**.

This time, the string `"Sai"` is **already present** in the pool.

So instead of creating another object, the JVM simply returns the reference to the existing string object.

The variable `s2` now points to the **same object** as `s1`.

```
String Constant Pool
```



## Step 3: Execute the third line

```
System.out.println(s1 == s2);
```

Here we are using the `==` operator.

The `==` operator **does not compare the contents of the strings**.

Instead, it compares whether **both variables refer to the same object in memory**.

In this example:

- `s1` points to the `"Sai"` object.
- `s2` also points to the **same** `"Sai"` object.

Since both variables refer to the same object, the expression

```
s1 == s2
```

evaluates to

```
true
```

## Interview Tip

What does `==` compare for objects?

**Answer:**

The `==` operator compares whether two reference variables are referring to the same object in memory. It does **not** compare the contents of the objects.